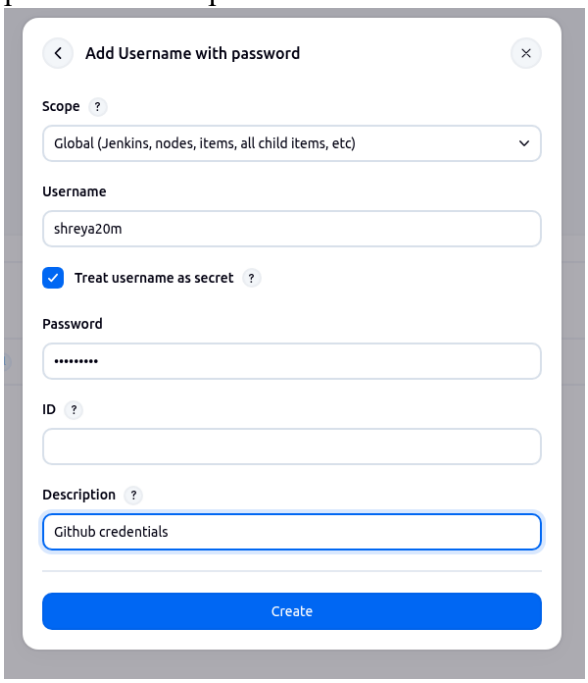
Continuous Integration (CI) Pipeline with Jenkins & Maven/Gradle

Step 1: Login to Jenkins and go to the Credentials page and click on add credentials
Step 2: Add new credentials in Jenkins for your CI pipeline, type such as "Username with password" for Git , enter details like username, password, set the scope to Global, and provide a description GitHub Credential.



Finally, click "Create" to securely store and use the credential in Jenkins pipelines.
Step 3: now open the terminal and type the command **cd MyMavenApp** to enter the folder and create a **Jenkinsfile** .Paste the code in the Jenkinsfile

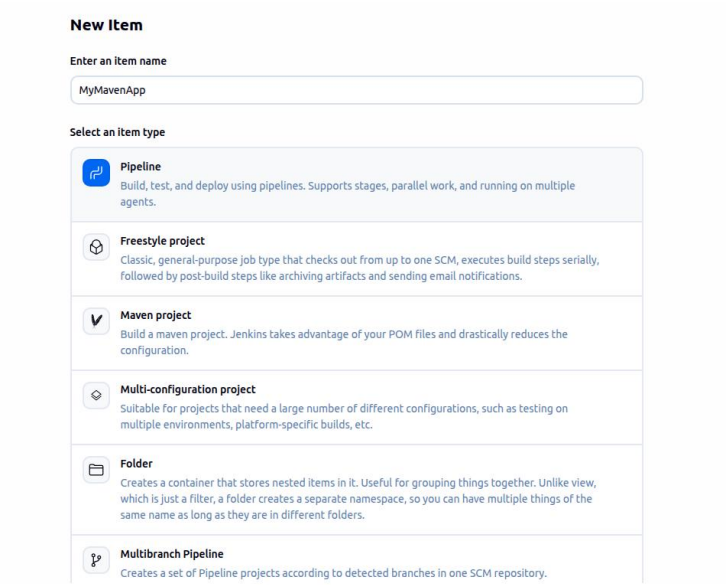
```
pipeline {
    agent any  // Use any available agent
    tools {
        maven 'Maven'  // Ensure this matches the name configured in Jenkins
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url: 'https://github.com/shreya20m /MyMavenApp.git'
            }
        }
        stage('Build') {
            steps {
```

```
            sh 'mvn clean package'  // Run Maven build
        }
    }
    stage('Test') {
        steps {
            sh 'mvn test'  // Run unit tests
        }
    }
    stage('Run Application') {
        steps {
            // Start the JAR application
            sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'
        }
    }
}
post {
    success {
        echo 'Build and deployment successful!'
    }
    failure {
        echo 'Build failed!'
    }}}
```

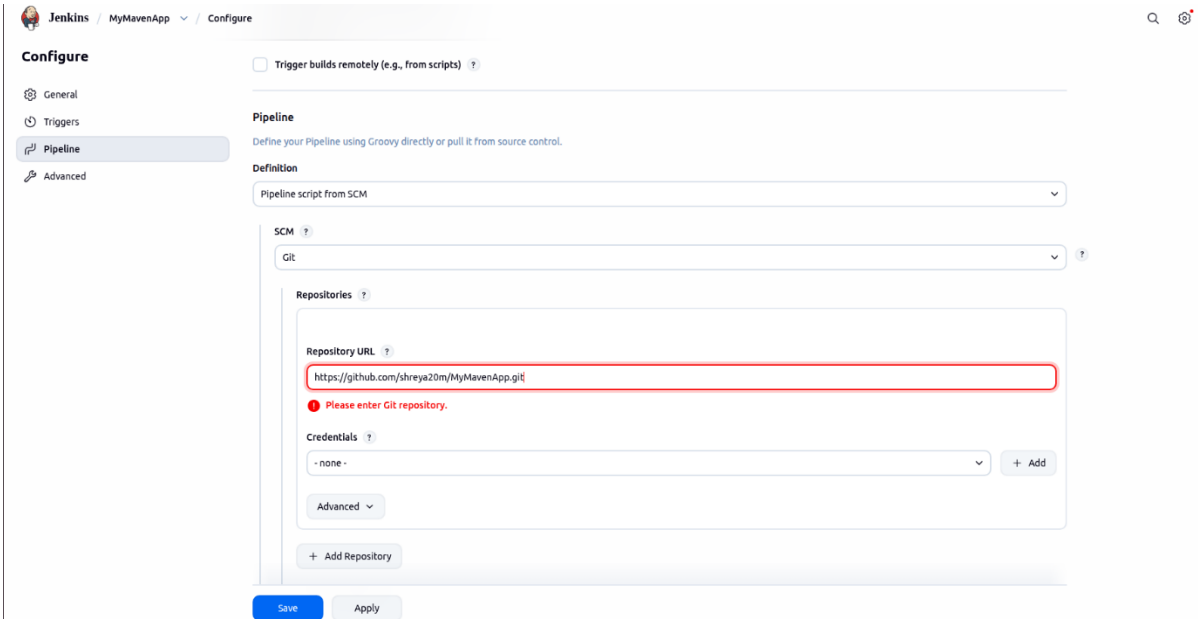
Then commit and push it to GitHub

```
shreya@ShreyaM:~/devops/MyMavenApp$ git remote set-url origin https://github.com
/shreya20m/MyMavenApp.git
shreya@ShreyaM:~/devops/MyMavenApp$ git push origin master
Username for 'https://github.com': shreya20m
Password for 'https://shreya20m@github.com':
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 2 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 1.39 KiB | 355.00 KiB/s, done.
Total 8 (delta 4), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com/shreya20m/MyMavenApp.git
   3532cc9..c8f81f4  master -> master
shreya@ShreyaM:~/devops/MyMavenApp$
```

Step 4:Go to Jenkins Dashboard and click on new item , enter the name as MyMavenApp and choose pipeline



Step 5: In MyMavenApp, go to the Pipeline section and under Definition, select Pipeline script from SCM. Choose git ,type git repository url choose git credential which we created. Then click Apply and Save to finalize the configuration.



Step 6: in Jenkins and click Build Now. if errors appear under Build, Test, or Run Application, open your GitHub account, go to the MyMavenApp repository, and edit the pom.xml file to update the Java version change from 1.7 to 11

MyMavenApp - Stage View

	Declarative: Checkout SCM	Declarative: Tool Install	Checkout	Build	Test	Run Application	Declarative: Post Actions
Average stage times:	3s	1s	2s	28s	439ms	174ms	263ms
#1 Apr 09 11:45 No Changes	3s	1s	2s	28s failed	439ms failed	174ms failed	263ms

After updating build again and check.

	Declarative: Checkout SCM	Declarative: Tool Install	Checkout	Build	Test	Run Application	Declarative: Post Actions
Average stage times: (full run time: ~5min 3s)	13s	3s	5s	1min 9s	3s	12s	605ms
#2 Apr 14 13:30 1 commit	13s	4s	6s	2min 18s	5s	4s	719ms

Gradle with Jenkins

Step 1: in the terminal type the code MyGradleApp to enter the folder .

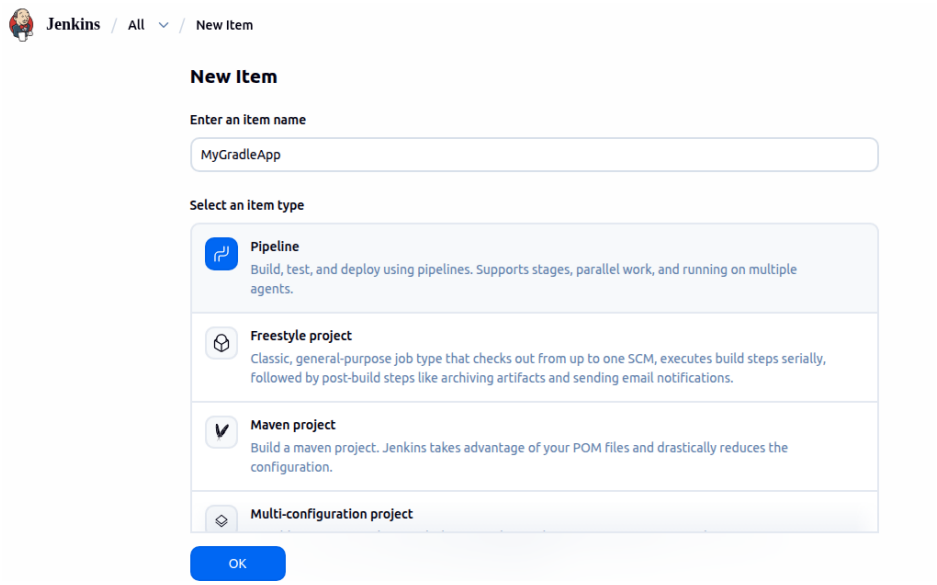
Type the command gedit JenkinsFile to open the Jenkinsfile and paste the code

```
pipeline {
    agent any  // Use any available agent

    tools {
        gradle 'Gradle'  // Ensure this matches the name configured in Jenkins
        jdk 'JDK'
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url: 'https://github.com/shreya20m/MyGradleApp.git'
            }
        }

        stage('Build') {
            steps {
                sh 'gradle build'  // Run Gradle build
            }
        }

        stage('Test') {
            steps {
                sh 'gradle test'  // Run unit tests
            }
        }





        stage('Run Application') {
```

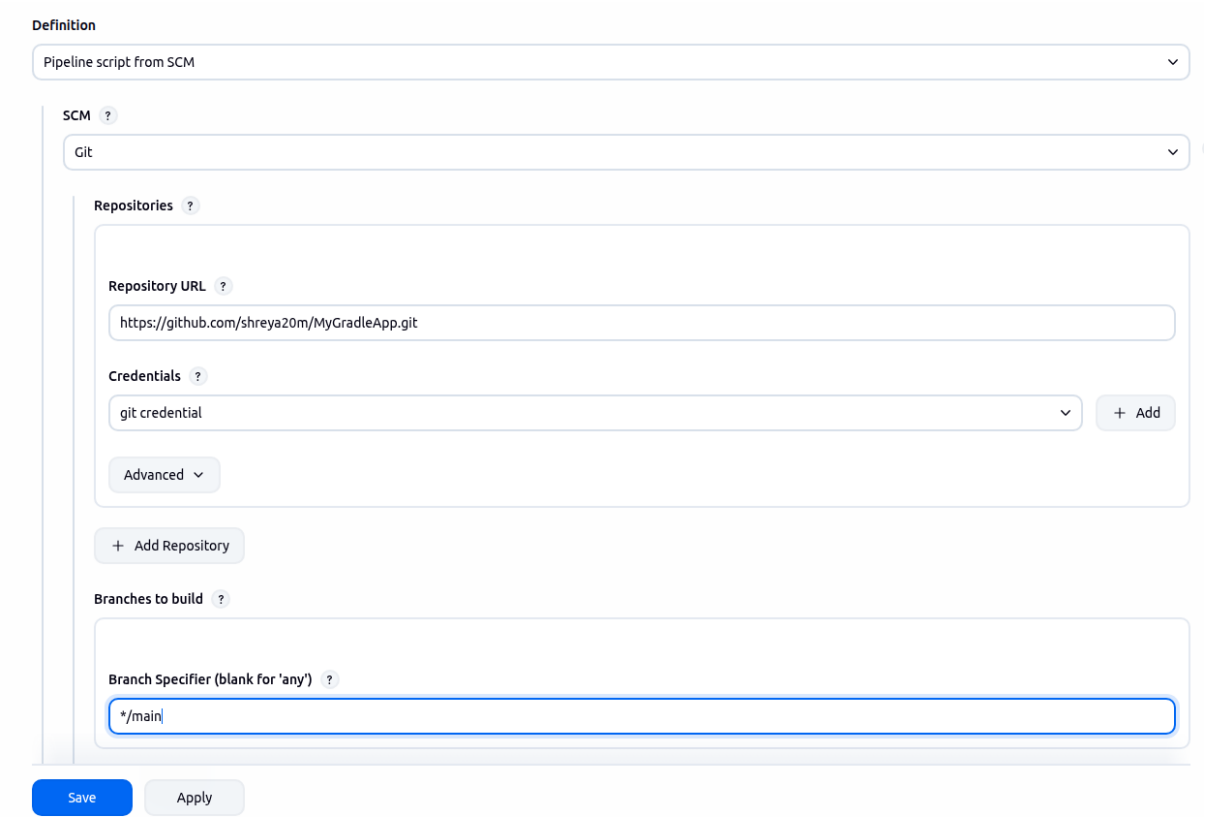
```
        steps {
            // Start the JAR application
            sh 'gradle run'
        }
    }



}

post {
    success {
        echo 'Build and deployment successful!'
    }
    failure {
        echo 'Build failed!'
    }
}
}
```

Step 2: in Jenkins Dasboard click on New Item

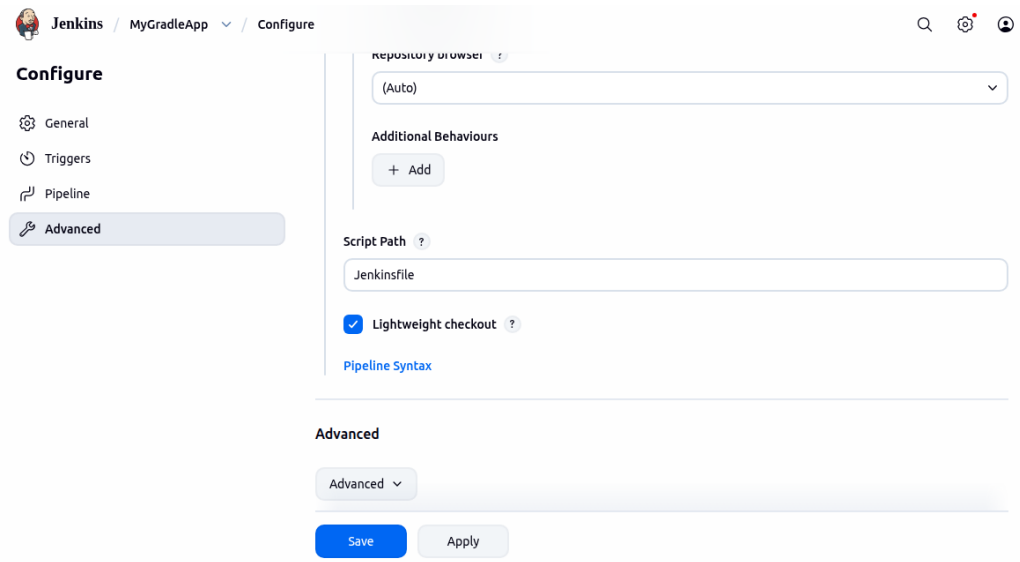
Give the name MyGradleApp and select Pipeline



In configure scroll down to Pipeline and choose pipeline script from SCM,give the repository url and choose GitHub Credentials

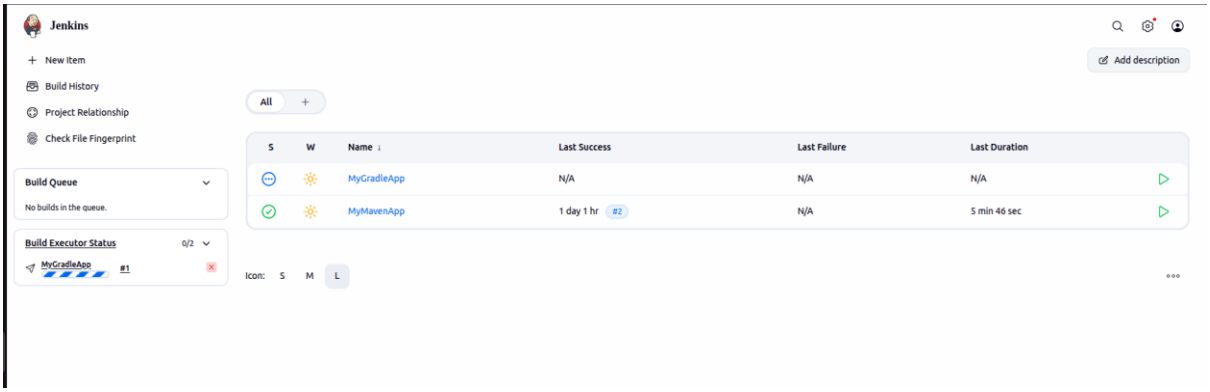


Then click on apply and save

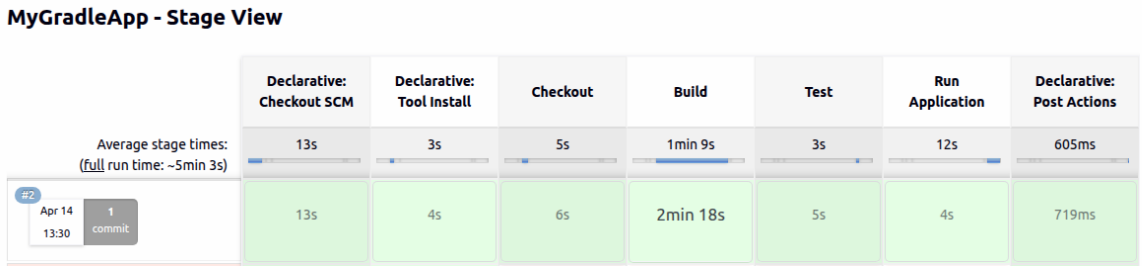


Step 3: go to Jenkins Dasboard select the MyGradleApp pipeline and select Build Now. After build select Full Stage View

Devops



Once the errors are corrected Build again



MyMavenToGradle in Jenkins:

Step 1: in the terminal type the code MavenToGradle to enter the folder .

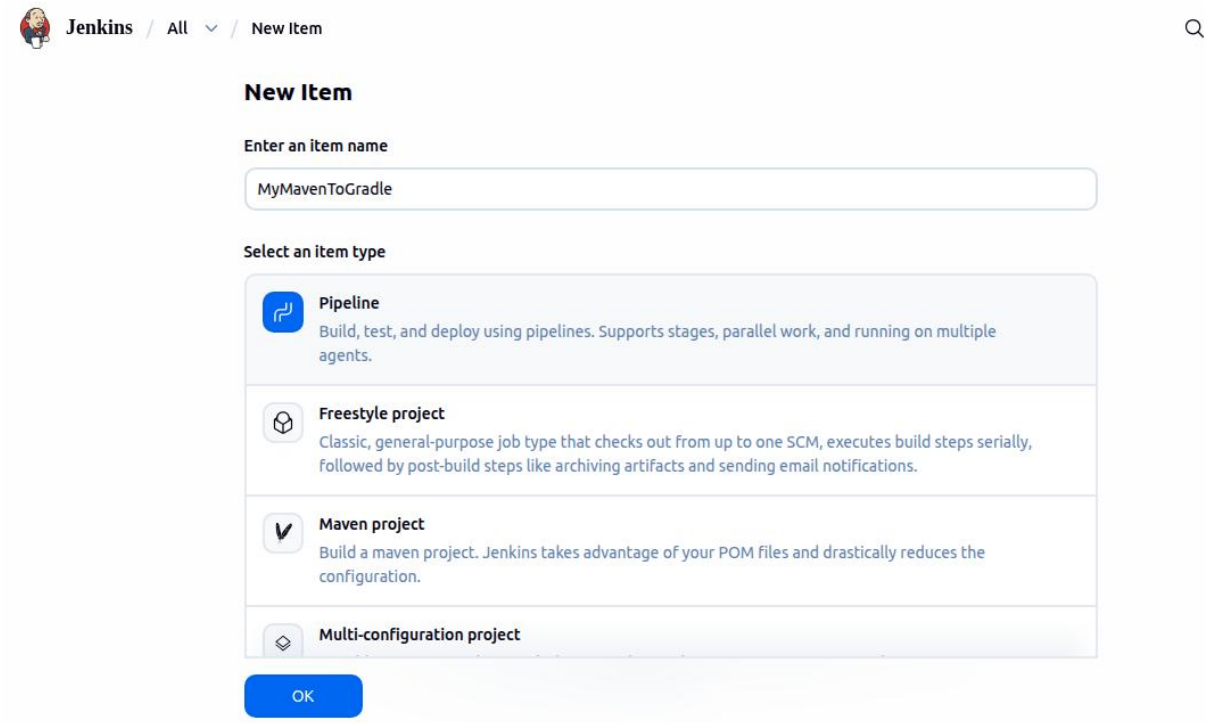
Type the command gedit JenkinsFile to open the Jenkinsfile and paste the code

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/shreya20m/maventogradle.git'
            }
        }

        stage('Build') {
            steps {
                sh 'chmod +x gradlew'
                sh './gradlew clean build'
            }
        }

        stage('Test') {
            steps {
                sh './gradlew test'
            }
        }

        stage('Run Application') {
            steps {
                sh './gradlew run'
```

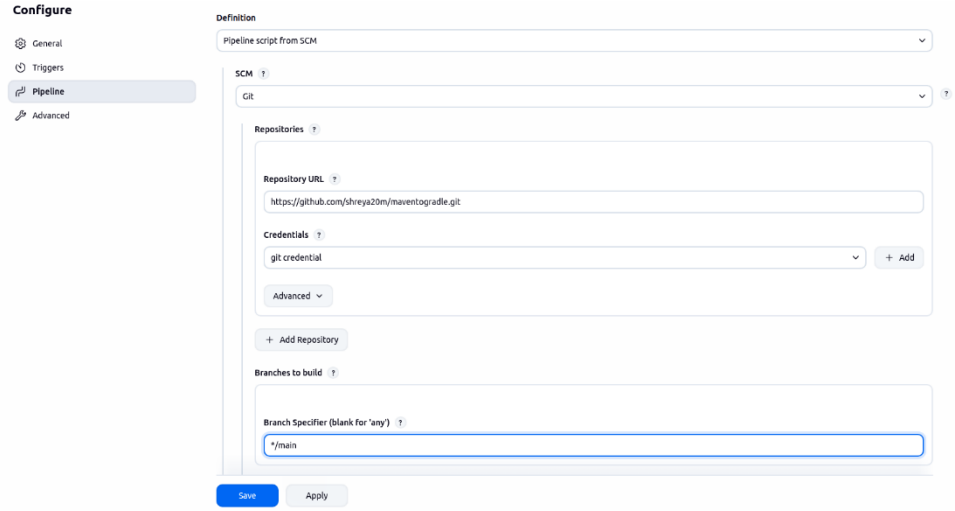
```
            }
        }
    }

    post {
        success {
            echo 'Build successful!'
        }
        failure {
            echo 'Build failed!'
        }
    }
}
```

Step 2: in Jenkins Dasboard click on New Item

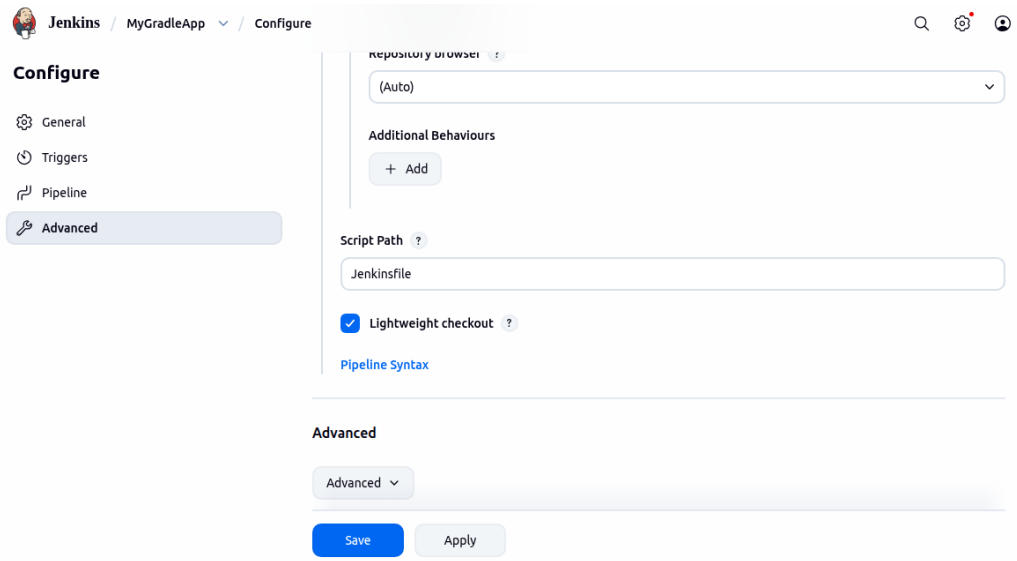
Give the name MyMavenToGradle and select Pipeline



In configure scroll down to Pipeline and choose pipeline script from SCM,give the repository url and choose GitHub Credential



Then click on apply and save



Step 3: go to Jenkins Dasboard select the MyMavenToGradle pipeline and select Build Now. After build select Full Stage View. Once the errors are corrected Build again

Devops

MyMavenToGradle - Stage View

	Declarative: Checkout SCM	Checkout	Build	Test	Run Application	Declarative: Post Actions
Average stage times: (full run time: ~4min 30s)	10s	12s	1min 22s	3s	2s	1s
#4 Apr 14 15:20 — 1 commit	5s	3s	3min 20s	9s	6s	453ms
#3 Apr 14 14:34 — 2 commits	17s	30s	18s failed	553ms failed	546ms failed	1s
#2 Apr 14 14:27 — 1 commit	6s	6s	14s failed	1s failed	1s failed	2s
#1 Apr 14 14:07 — No Changes	10s	8s	1min 34s failed	2s failed	890ms failed	3s

MyMavenGuava in Jenkins:

Step 1: in the terminal type the code MyMavenGuava to enter the folder .

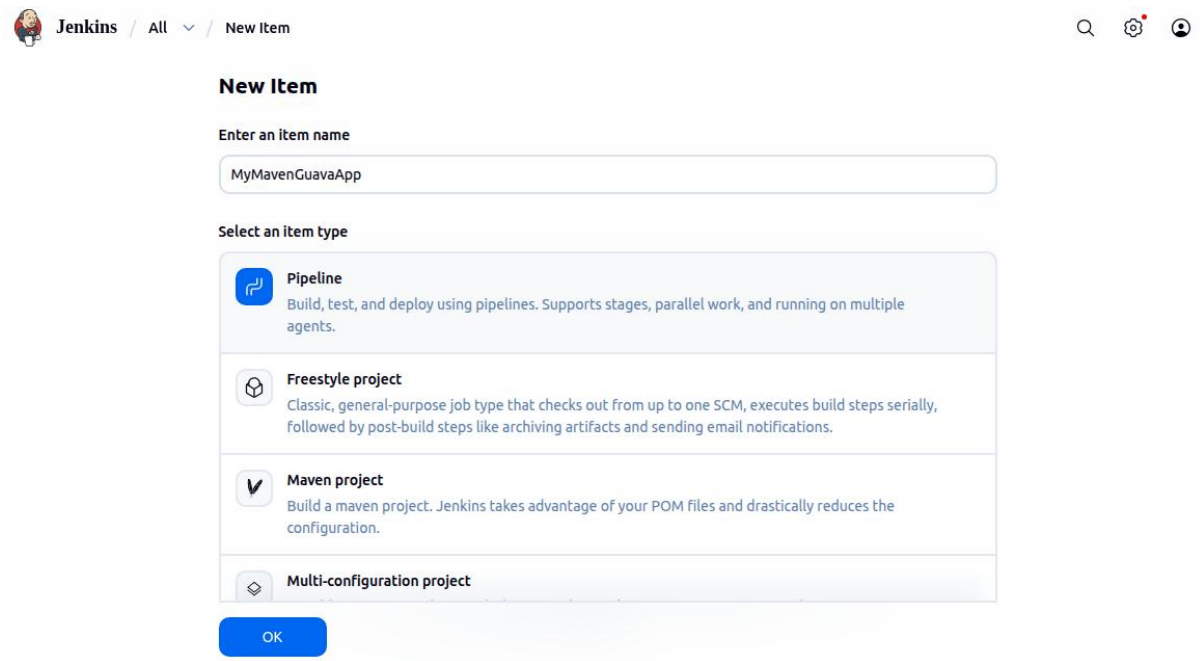
Type the command gedit JenkinsFile to open the Jenkinsfile and paste the code

```
pipeline {
    agent any

    tools {
        maven 'Maven'
    }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url: 'https://github.com/shreya20m/MyMavenGuavaApp.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean package -DskipTests'
            }
        }

        stage('Run Application') {
            steps {
                sh 'java -jar target/MyMavenGuavaApp-1.0-SNAPSHOT.jar'
            }
        }
    }

    post {
        success {
            echo 'Build and deployment successful!'
```

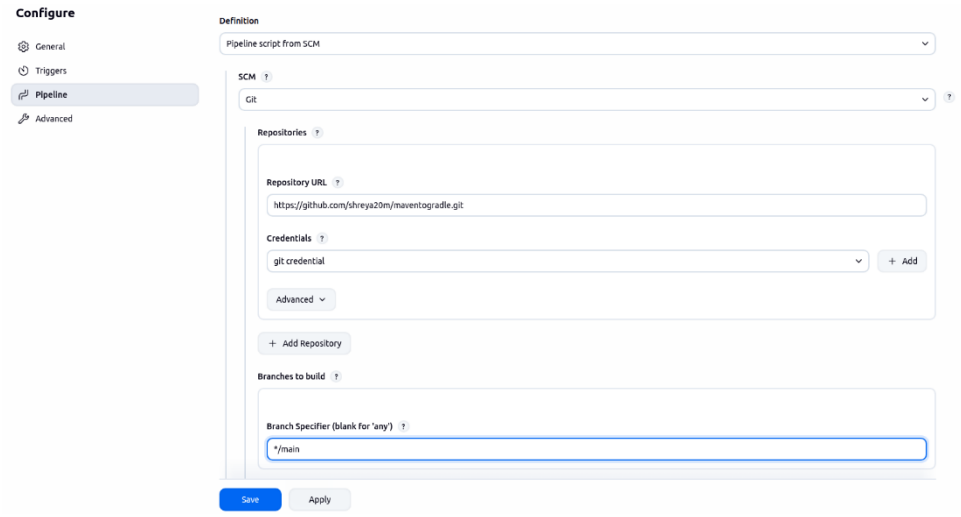
```
        }
        failure {
            echo 'Build failed!'
        }
    }
}
```

Step 2: in Jenkins Dasboard click on New Item

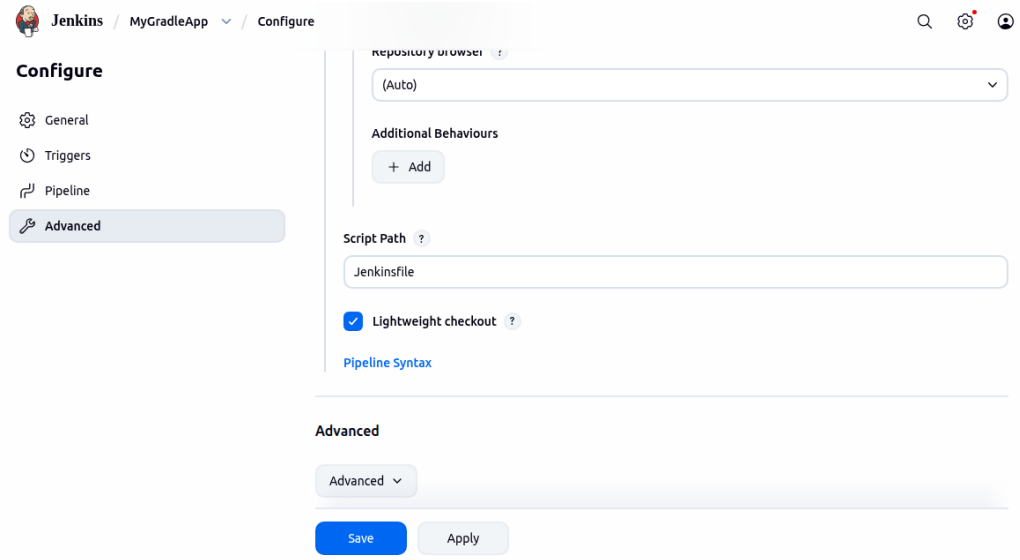
Give the name MyMavenGuava and select Pipeline



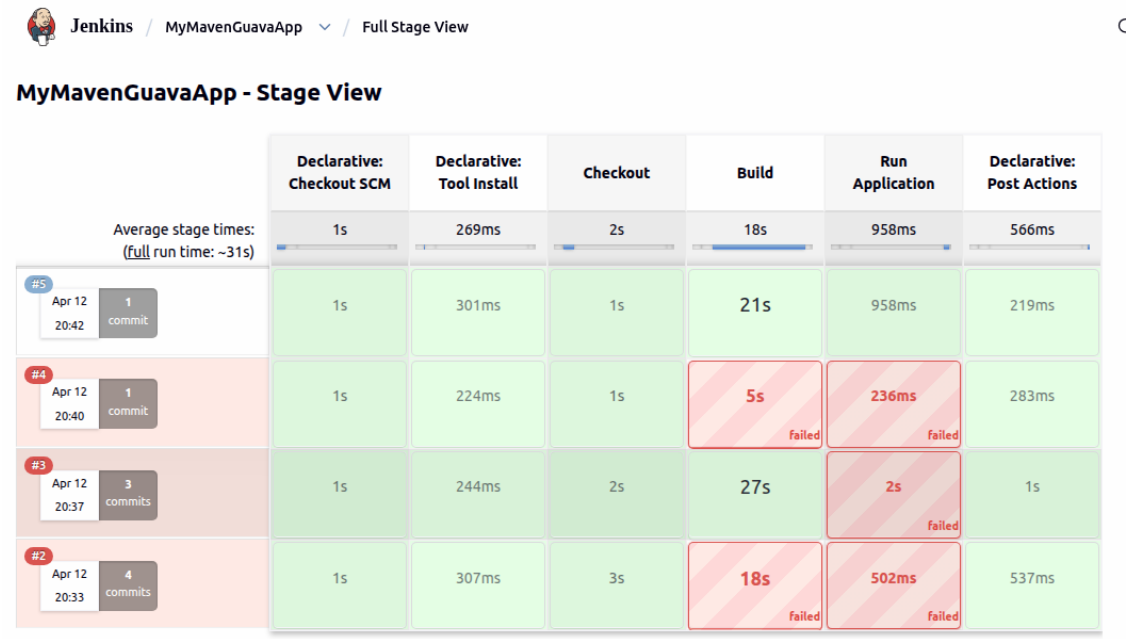
In configure scroll down to Pipeline and choose pipeline script from SCM,give the repository url and choose GitHub Credential



Then click on apply and save



Step 3: go to Jenkins Dasboard select the MyMavenGuava pipeline and select Build Now. After build select Full Stage View. Once the errors are corrected Build again



SauceDemo in jenkins:

Step 1: in the terminal type the code saucedemo to enter the folder .

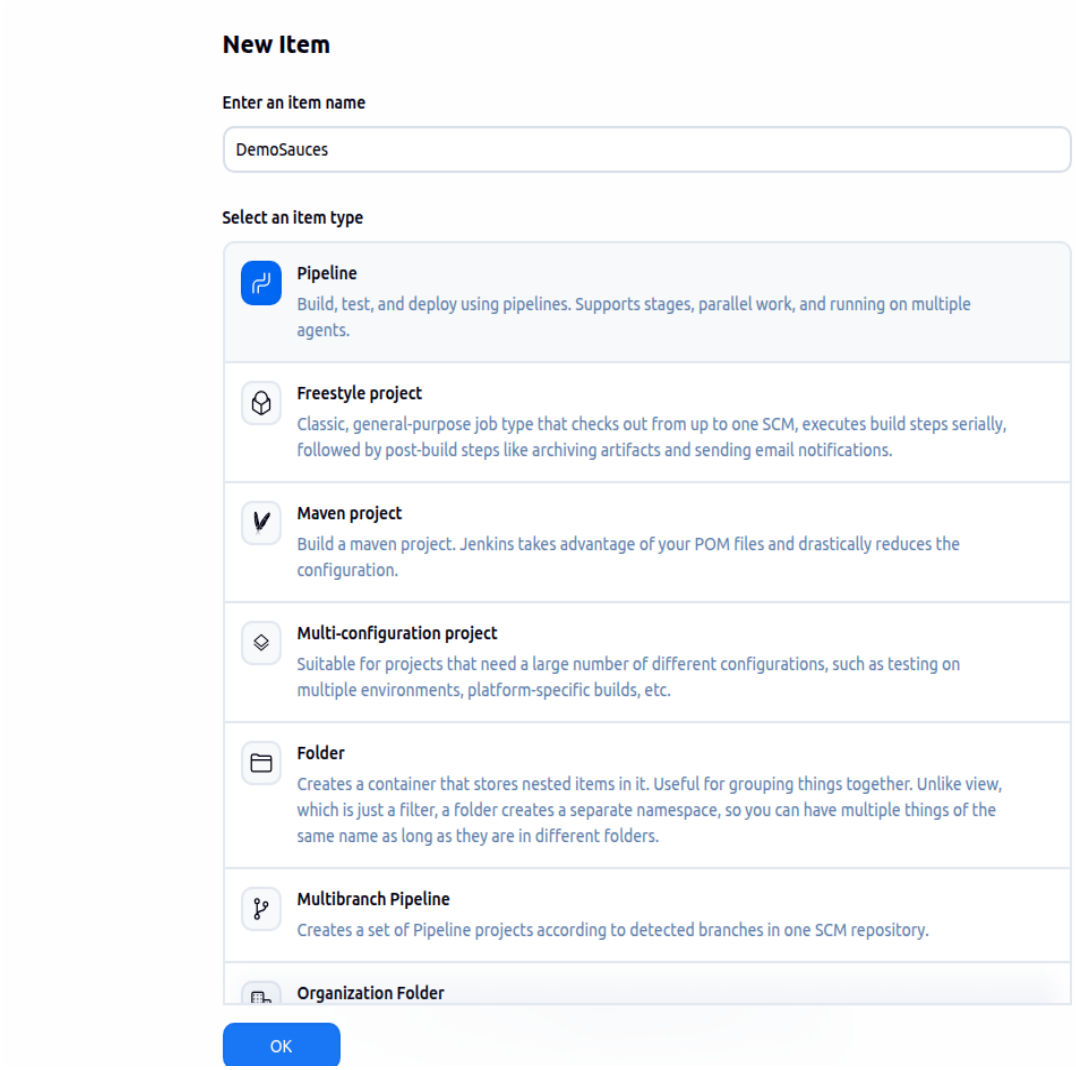
Type the command gedit JenkinsFile to open the Jenkinsfile and paste the code

```
pipeline {
    agent any

    tools {
        maven 'Maven'
    }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url: 'https://github.com/shreya20m/saucedemo.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean package -DskipTests'
            }
        }

        stage('Run Application') {
            steps {
                sh 'java -jar target/saucedemo-1.0-SNAPSHOT.jar'
            }
        }
    }

    post {
        success {
            echo 'Build and deployment successful!'
        }
```

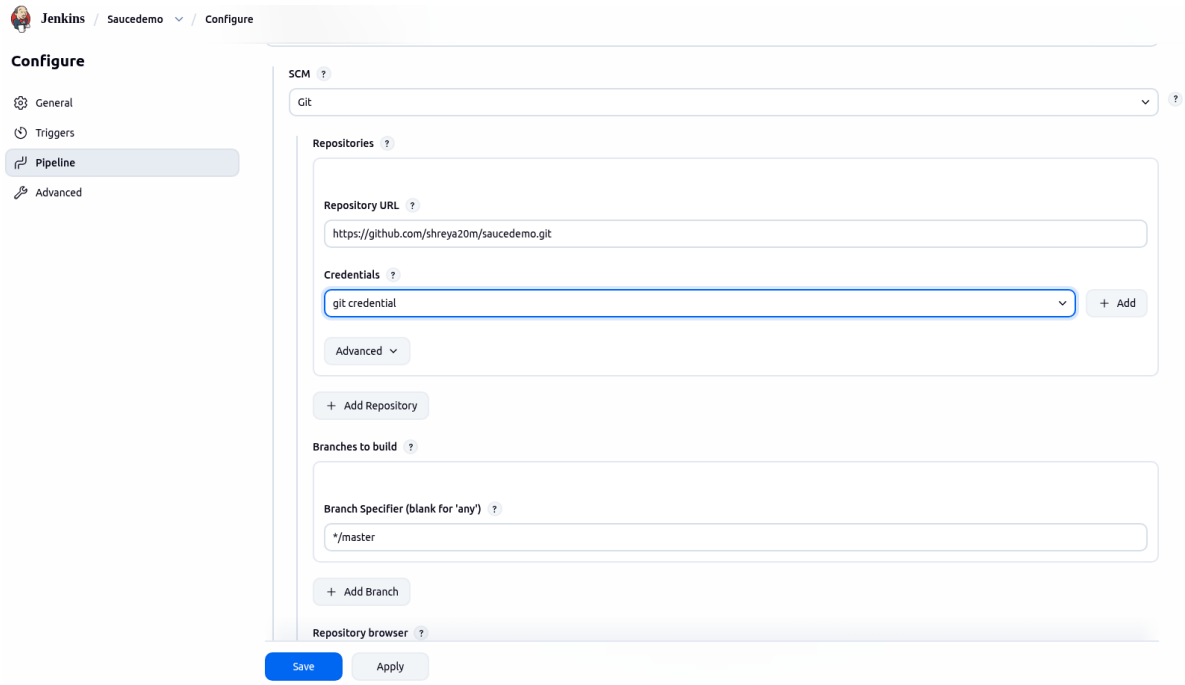
```
        failure {
            echo 'Build failed!'
        }
    }
}
```

Step 2: in Jenkins Dasboard click on New Item

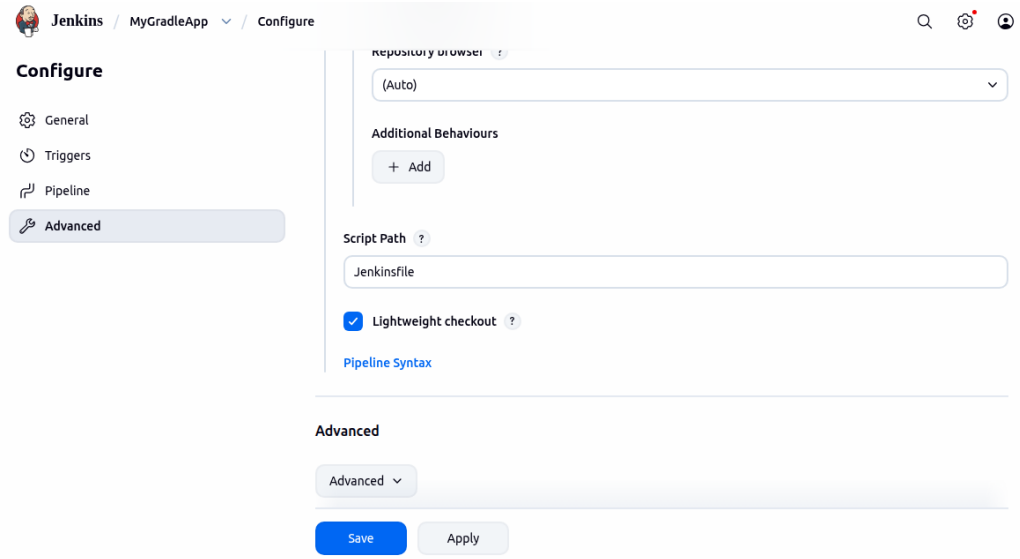
Give the name Saucedemo and select Pipeline



In configure scroll down to Pipeline and choose pipeline script from SCM,give the repository url and choose GitHub Credential

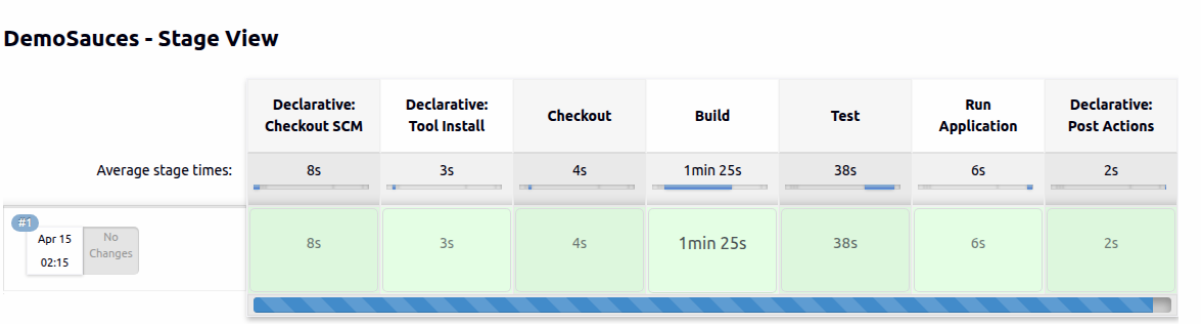


Then click on apply and save



Step 3: go to Jenkins Dasboard select the Saucedemo pipeline and select Build Now. After build select Full Stage View. Once the errors are corrected Build again

Devops


DemoSauces - Stage View
Declarative: Checkout SCM
Declarative: Tool Install
Checkout
Build
Test
Run Application
Declarative: Post Actions
Average stage times:
8s
3s
4s
1min 25s
38s
6s
2s
#1
Apr 15
02:15
No Changes
8s
3s
4s
1min 25s
38s
6s
2s

AutomationExercise in jenkins:

Step 1: in the terminal type the code AutomationExercise to enter the folder .

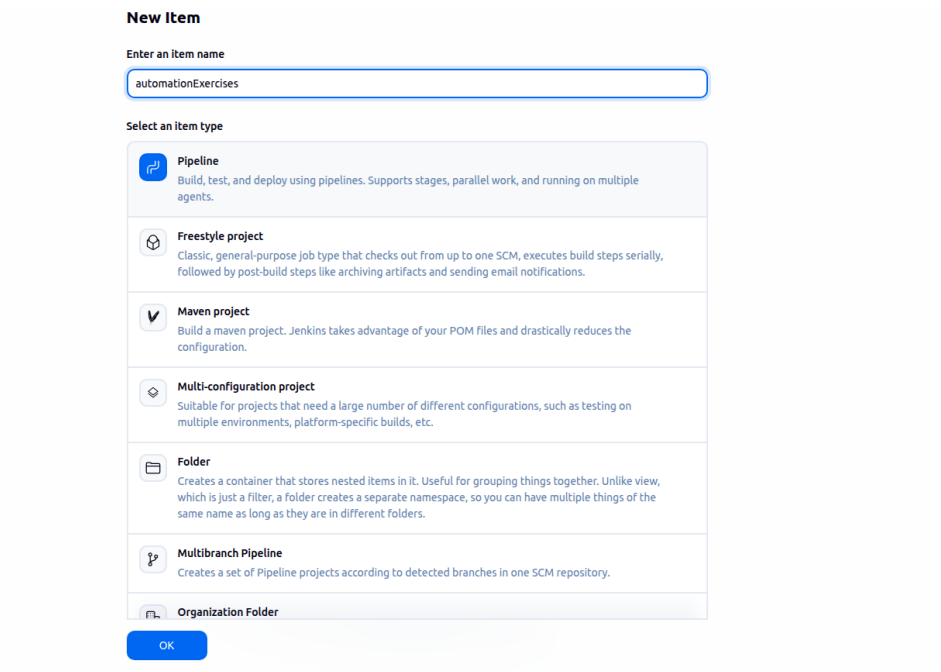
Type the command gedit JenkinsFile to open the Jenkinsfile and paste the code

```
pipeline {
    agent any

    tools {
        maven 'Maven'
    }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url: 'https://github.com/shreya20m/automationexercise.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean package -DskipTests'
            }
        }

        stage('Run Application') {
            steps {
                sh 'java -jar target/automationexercise -1.0-SNAPSHOT.jar'
            }
        }
    }

    post {
        success {
            echo 'Build and deployment successful!'
        }
```

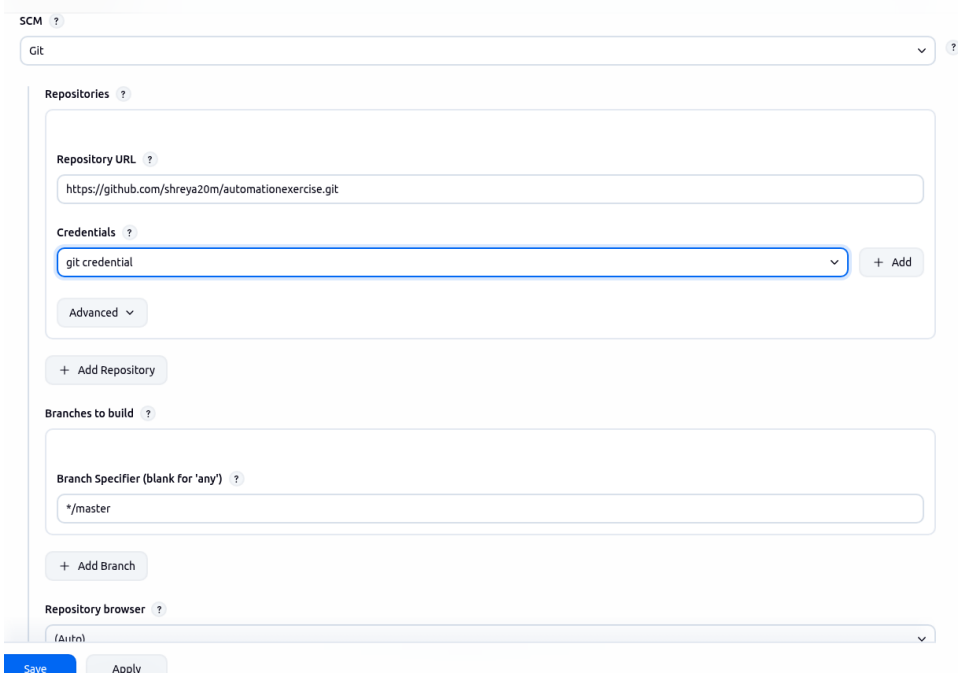
```
        failure {
            echo 'Build failed!'
        }
    }
}
```

Step 2: in Jenkins Dasboard click on New Item

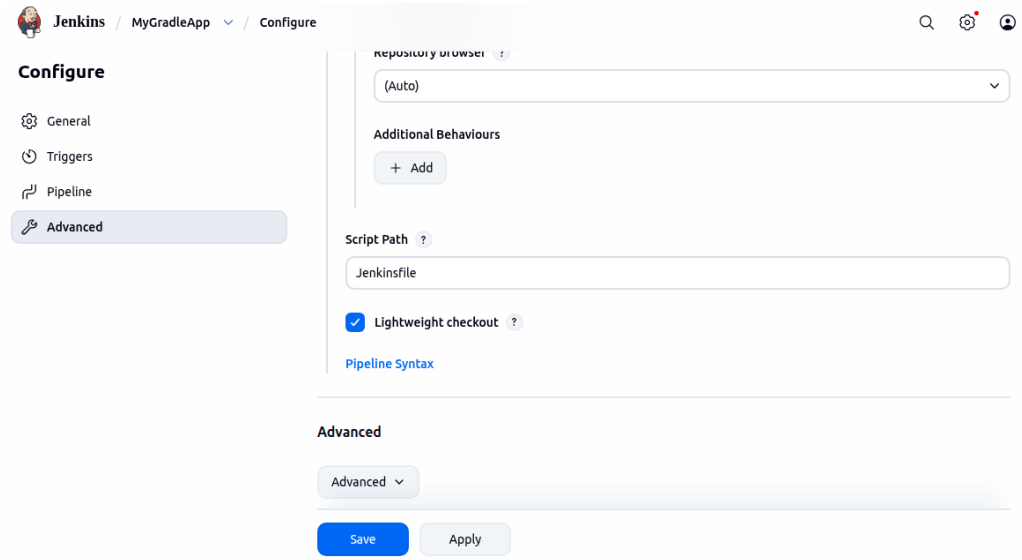
Give the name AutomationExercise and select Pipeline



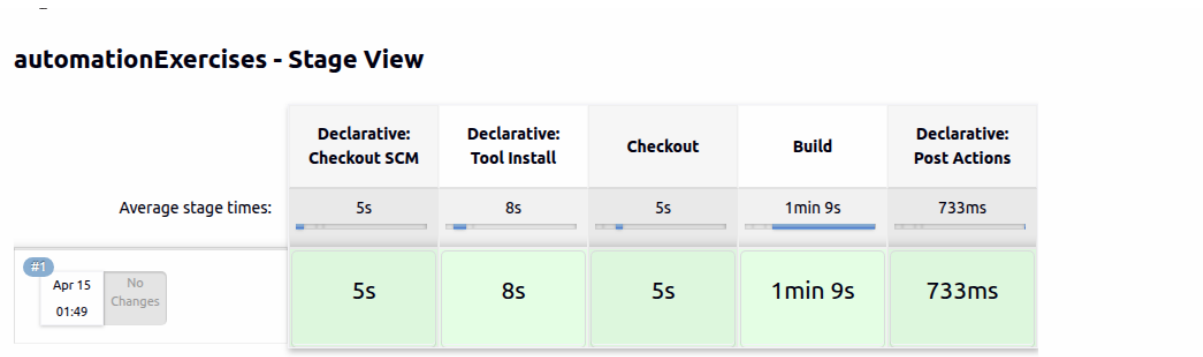
In configure scroll down to Pipeline and choose pipeline script from SCM,give the repository url and choose GitHub Credential



Then click on apply and save



Step 3: go to Jenkins Dasboard select the PractiseTest pipeline and select Build Now. After build select Full Stage View. Once the errors are corrected Build again



PracticeTest in jenkins:

Step 1: in the terminal type the code practicetest to enter the folder .

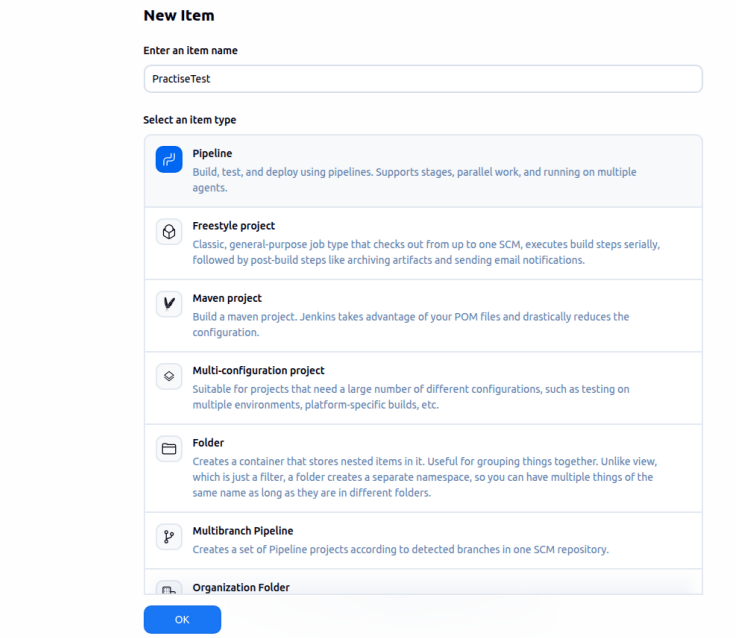
Type the command gedit JenkinsFile to open the Jenkinsfile and paste the code

```
pipeline {
   agent any

   tools {
      maven 'Maven'
   }

   stages {
      stage('Checkout') {
         steps {
            git branch: 'master', url: 'https://github.com/shreya20m/practicetest.git'
         }
```

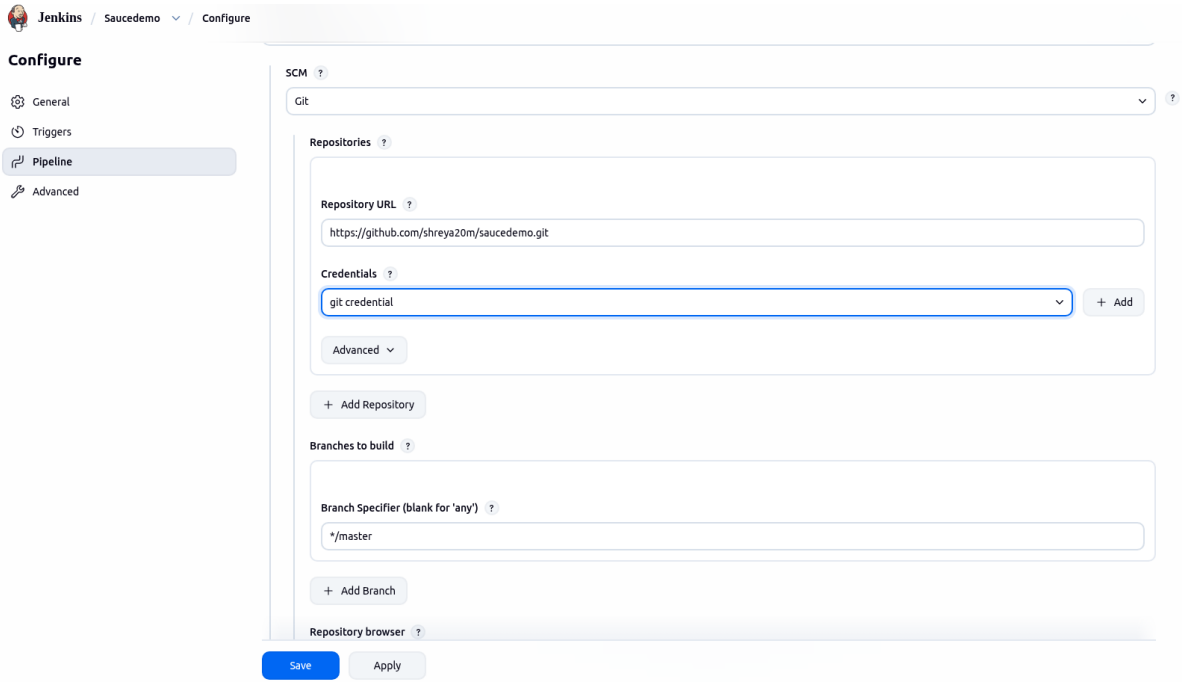
```
        }

        stage('Build') {
            steps {
                sh 'mvn clean package -DskipTests'
            }
        }

        stage('Run Application') {
            steps {
                sh 'java -jar target/practisetest-1.0-SNAPSHOT.jar'
            }
        }
    }

    post {
        success {
            echo 'Build and deployment successful!'
        }
        failure {
            echo 'Build failed!'
        }
    }
}
```

Step 2: in Jenkins Dasboard click on New Item

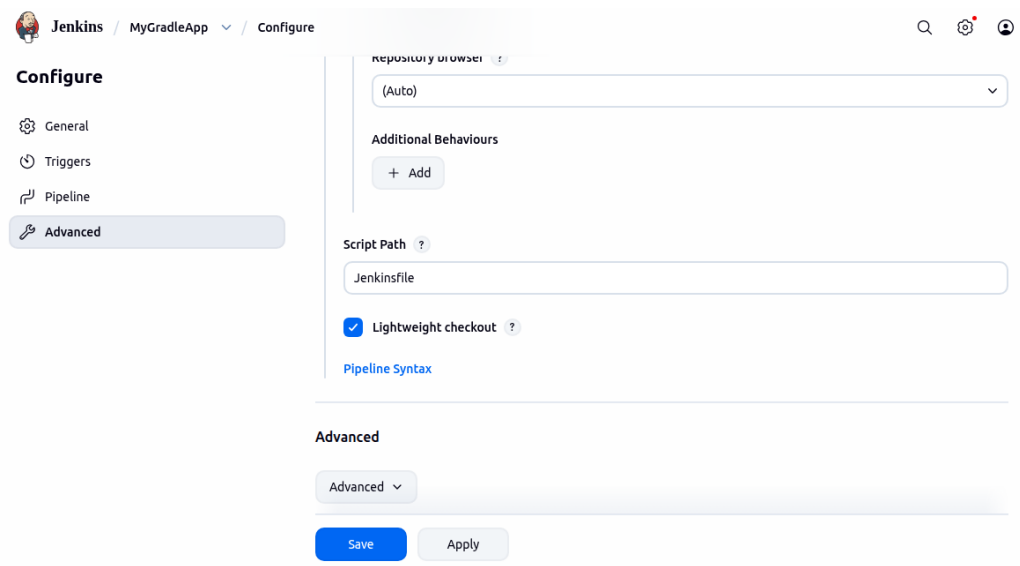
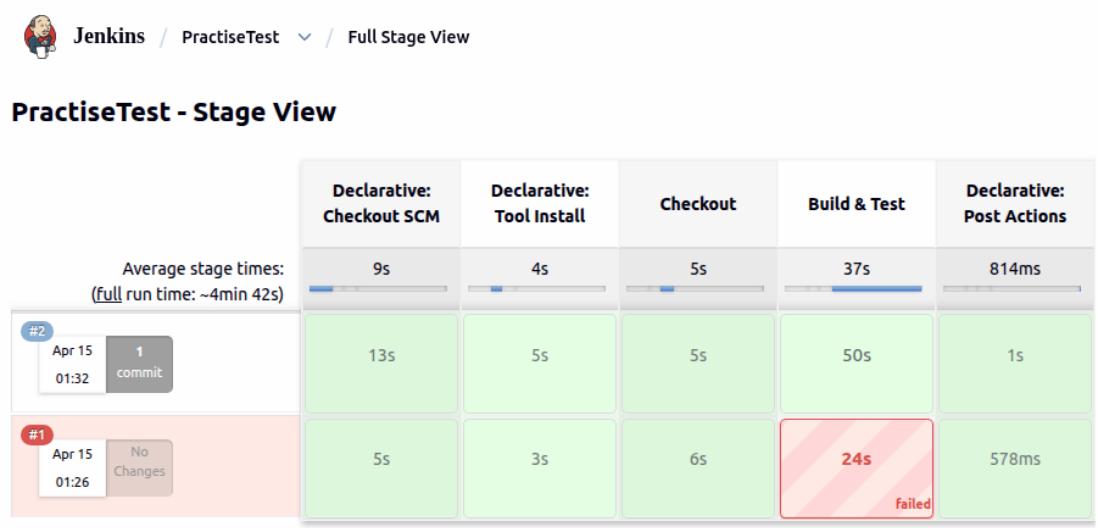
Give the name practisetest and select Pipeline



In configure scroll down to Pipeline and choose pipeline script from SCM,give the repository url and choose GitHub Credential



Then click on apply and save



Step 3: go to Jenkins Dasboard select the PractiseTest pipeline and select Build Now. After build select Full Stage View. Once the errors are corrected Build again